

Business Problem: A car rental company wants to analyze the rental durations of its

customers to understand the typical rental period and optimize its pricing and fleet

management strategies. Data: Let's consider the rental durations (in days) for a sample of 50 customers:

3, 2, 5, 4, 7, 2, 3, 3, 1, 6, 4, 2, 3, 5, 2, 4, 2, 1, 3, 5, 6, 3, 2, 1, 4,
2, 4, 5, 3, 2, 7, 2, 3,

4, 5, 1, 6, 2, 4, 3, 5, 3, 2, 4, 2, 6, 3, 2, 4, 5

Question:

a. Mean: What is the average rental duration for customers at the car rental company?

b. Median: What is the typical or central rental duration experienced by customers?

c. Mode: Are there any recurring or most frequently occurring rental durations for customers?

```
import numpy as np
from scipy import stats
rental_days = np.array([3, 2, 5, 4, 7, 2, 3, 3, 1, 6, 4, 2, 3, 5, 2, 4,
2, 1, 3, 5, 6, 3, 2, 1, 4, 2, 4, 5, 3, 2, 7, 2, 3,
4, 5, 1, 6, 2, 4, 3, 5, 3, 2, 4, 2, 6, 3, 2, 4, 5])
```

```
## a)
```

```
Mean = np.mean(rental_days)
print(f"The average rental duration for customers at the car rental  
company : {Mean}")
```

```
The average rental duration for customers at the car rental company :  
3.44
```

```
## b)
Median = np.median(rental_days)
print(f"The central rental duration experienced by customers :
{Median}")

The central rental duration experienced by customers : 3.0

## c)
from scipy import stats
Mode = stats.mode(rental_days)
print(f"Most frequently occurring rental durations for customers :
{Mode}")

Most frequently occurring rental durations for customers :
ModeResult(mode=2, count=13)
```

Question 2

An e-commerce platform wants to analyze the delivery times of its shipments to

understand the variability in order fulfillment and optimize its logistics operations.

Data:

Let's consider the delivery times (in days) for a sample of 50 shipments:

3, 5, 2, 4, 6, 2, 3, 4, 2, 5,
7, 2, 3, 4, 2, 4, 2, 3, 5, 6,
3, 2, 1, 4, 2, 4, 5, 3, 2, 7,
2, 3, 4, 5, 1, 6, 2, 4, 3, 5,
3, 2, 4, 2, 6, 3, 2, 4, 5, 3

Questions:

- Range: What is the range of the delivery times?
- Variance: What is the variance of the delivery times?
- Standard Deviation: What is the standard deviation of the delivery times?

```
delivery = np.array([3, 5, 2, 4, 6, 2, 3, 4, 2, 5,  
                    7, 2, 3, 4, 2, 4, 2, 3, 5, 6,  
                    3, 2, 1, 4, 2, 4, 5, 3, 2, 7,  
                    2, 3, 4, 5, 1, 6, 2, 4, 3, 5,  
                    3, 2, 4, 2, 6, 3, 2, 4, 5, 3])  
  
# a)  
Range = delivery.max()-delivery.min()  
print(f"Range of delivery times : {Range}")
```

```
Range of delivery times : 6
```

```
# b)
```

```
Variance = np.var(delivery)
```

```
print(f"Variance of delivery times : {Variance:.3f}")
```

```
Variance of delivery times : 2.290
```

```
# c)
```

```
Standard_dev = np.std(delivery)
```

```
print(f"Standard deviation of delivery times : {Standard_dev:.3f}")
```

```
Standard deviation of delivery times : 1.513
```

Question 3

A company wants to analyze the monthly revenue generated by one of its products

to understand its performance and variability.

Data: Let's consider the monthly revenue (in thousands of dollars) for the past 12 months:

\$120, \$150, \$110, \$135, \$125, \$140, \$130, \$155, \$115, \$145, \$135, \$130

Questions:

a. Measure of Central Tendency: What is the average monthly revenue for the product?

b. Measure of Dispersion: What is the range of monthly revenue for the product?

```
revenue = np.array([120, 150, 110, 135, 125, 140, 130, 155, 115, 145, 135, 130])
```

```
## a) Measure of central Tendency(Mean)
```

```
Avg_revenue = np.mean(revenue)
```

```
print(f"Average monthly revenue for the product : {Avg_revenue}")
```

```
Average monthly revenue for the product : 132.5
```

```
## b) Measure of Dispersion(Range)
Range_Revenue = revenue.max()-revenue.min()
print(f"Range of monthly revenue for the product : {Range_Revenue}")

Range of monthly revenue for the product : 45
```

Question 4

A transportation company wants to analyze the fuel efficiency of its vehicle fleet to

identify any variations across different vehicle models.

Data: Let's consider the fuel efficiency (in miles per gallon, mpg) for a sample of 50 vehicles:

Model A: 30, 32, 33, 28, 31, 30, 29, 30, 32, 31,

Model B: 25, 27, 26, 23, 28, 24, 26, 25, 27, 28,

Model C: 22, 23, 20, 25, 21, 24, 23, 22, 25, 24,

Model D: 18, 17, 19, 20, 21, 18, 19, 17, 20, 19,

Model E: 35, 36, 34, 35, 33, 34, 32, 33, 36, 34

Questions:

a. Measure of Central Tendency: What is the average fuel efficiency for each vehicle model?

b. Measure of Dispersion: What is the range of fuel efficiency for each vehicle model?

c. Measure of Dispersion: What is the variance of the fuel efficiency for each vehicle model?

```
Model_A = np.array([30, 32, 33, 28, 31, 30, 29, 30, 32, 31])
Model_B = np.array([25, 27, 26, 23, 28, 24, 26, 25, 27, 28])
Model_C = np.array([22, 23, 20, 25, 21, 24, 23, 22, 25, 24,])
```

```
Model_D = np.array([18, 17, 19, 20, 21, 18, 19, 17, 20, 19])
Model_E = np.array([35, 36, 34, 35, 33, 34, 32, 33, 36, 34])
```

```
## a)
```

```
M1 = np.mean(Model_A)
M2 = np.mean(Model_B)
M3 = np.mean(Model_C)
M4 = np.mean(Model_D)
M5 = np.mean(Model_E)
```

```
print(f"Fuel Efficiency of A : {M1}")
print(f"Fuel Efficiency of B : {M2}")
print(f"Fuel Efficiency of C : {M3}")
print(f"Fuel Efficiency of D : {M4}")
print(f"Fuel Efficiency of E : {M5}")
```

```
Fuel Efficiency of A : 30.6
Fuel Efficiency of B : 25.9
Fuel Efficiency of C : 22.9
Fuel Efficiency of D : 18.8
Fuel Efficiency of E : 34.2
```

```
## b)
```

```
r1 = Model_A.max()-Model_A.min()
r2 = Model_B.max()-Model_B.min()
r3 = Model_C.max()-Model_C.min()
r4 = Model_D.max()-Model_D.min()
r5 = Model_E.max()-Model_E.min()
```

```
print(f"Range of fuel efficiency of A :{r1}")
print(f"Range of fuel efficiency of B :{r2}")
print(f"Range of fuel efficiency of C :{r3}")
print(f"Range of fuel efficiency of D :{r4}")
print(f"Range of fuel efficiency of E :{r5}")
```

```
Range of fuel efficiency of A :5
Range of fuel efficiency of B :5
Range of fuel efficiency of C :5
Range of fuel efficiency of D :4
Range of fuel efficiency of E :4
```

```
## c)
```

```
Var1 = np.var(Model_A)
Var2 = np.var(Model_B)
Var3 = np.var(Model_C)
Var4 = np.var(Model_D)
Var5 = np.var(Model_E)
```

```
print(f"variance of the fuel efficiency for A : {Var1:.3f}")
print(f"variance of the fuel efficiency for B : {Var2:.3f}")
```

```
print(f"variance of the fuel efficiency for C : {Var3:.3f}")
print(f"variance of the fuel efficiency for D : {Var4:.3f}")
print(f"variance of the fuel efficiency for E : {Var5:.3f}")
```

```
variance of the fuel efficiency for A : 2.040
variance of the fuel efficiency for B : 2.490
variance of the fuel efficiency for C : 2.490
variance of the fuel efficiency for D : 1.560
variance of the fuel efficiency for E : 1.560
```

Question 5

A manufacturing company wants to analyze the defect rates of its production line to

identify the frequency of different types of defects.

Data: Let's consider the types of defects and their corresponding frequencies

observed in a sample of 200 products:

Defect Type: A, B, C, D, E, F, G

Frequency: 30, 40, 20, 10, 45, 25, 30

Questions:

a. Bar Chart: Create a bar chart to visualize the frequency of different defect types.

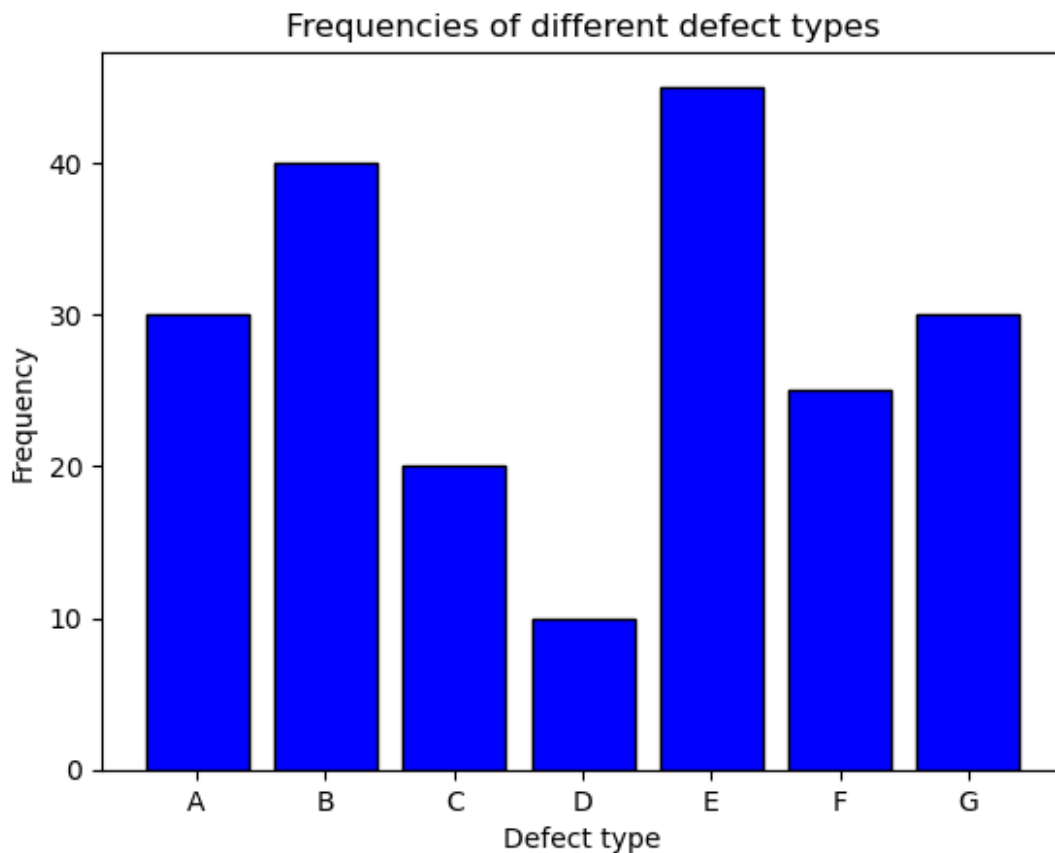
b. Most Common Defect: Which defect type has the highest frequency?

c. Histogram: Create a histogram to represent the defect frequencies.

```
import matplotlib.pyplot as plt

defect_types = ['A', 'B', 'C', 'D', 'E', 'F', 'G']
Frequencies = [30, 40, 20, 10, 45, 25, 30]
```

```
# a
plt.bar(defect_types,Frequencies,color='blue', edgecolor= 'black')
plt.title('Frequencies of different defect types')
plt.xlabel('Defect type')
plt.ylabel('Frequency')
plt.show()
```

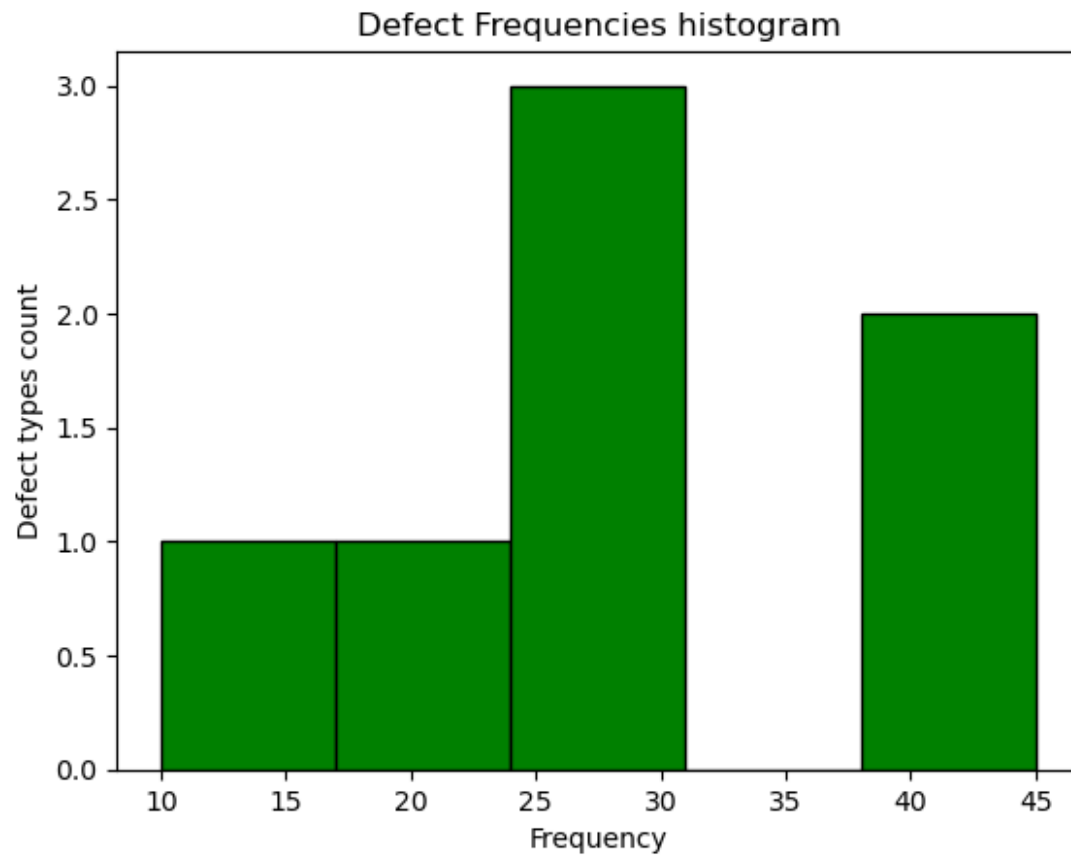


```
# b
max_frequency = max(Frequencies)
idx = Frequencies.index(max_frequency)
most_common_dfct= defect_types[idx]
print(f"The most common defect is Type {most_common_dfct} with a frequency of {max_frequency}.")
```

The most common defect is Type E with a frequency of 45.

```
# c
plt.hist(Frequencies, bins = 5, color = 'green', edgecolor = 'black')
plt.title('Defect Frequencies histogram')
plt.xlabel('Frequency')
plt.ylabel('Defect types count')

plt.show()
```

Question 6

A survey was conducted to analyze the satisfaction ratings of customers on a scale of 1 to 5 for a specific product.

Data: Let's consider the satisfaction ratings from 200 customers:

Ratings:

4, 5, 3, 4, 4, 3, 2, 5, 4, 3, 5, 4, 2, 3, 4, 5, 3, 4, 5, 3, 4, 3, 2, 4, 5,
3, 4, 5, 4, 3, 3, 4, 5,
2, 3, 4, 4, 3, 5, 4, 3, 4, 5, 4, 2, 3, 4, 5, 3, 4, 5, 4, 3, 4, 5, 3, 4, 5,
4, 3, 3, 4, 5, 2, 3, 4,
4, 3, 5, 4, 3, 4, 5, 4, 2, 3, 4, 5, 3, 4, 5, 4, 3, 4, 5, 3, 4, 5, 4, 3, 3,
4, 5, 2, 3, 4, 4, 3, 5,

4

Questions:

a. Skewness: Calculate the skewness of the satisfaction ratings.

b. Kurtosis: Calculate the kurtosis of the satisfaction ratings.

c. Interpretation: Based on the skewness and kurtosis values, what can be inferred about the satisfaction ratings distribution?

```
ratings = np.array([4, 5, 3, 4, 4, 3, 2, 5, 4, 3, 5, 4, 2, 3, 4, 5, 3,  
4, 5, 3, 4, 3, 2, 4, 5, 3, 4, 5, 4, 3, 3, 4, 5,  
2, 3, 4, 4, 3, 5, 4, 3, 4, 5, 4, 2, 3, 4, 5, 3, 4,
```

```
5, 4, 3, 4, 5, 3, 4, 5, 4, 3, 3, 4, 5, 2, 3, 4,
    4, 3, 5, 4, 3, 4, 5, 4, 2, 3, 4, 5, 3, 4, 5, 4, 3,
4, 5, 3, 4, 5, 4, 3, 3, 4, 5, 2, 3, 4, 4, 3, 5,
    4])
```

```
# a
```

```
from scipy.stats import skew
```

```
skewness = skew(ratings)
```

```
print(f"The skewness of the satisfaction ratings : {skewness:.3f}")
```

```
The skewness of the satisfaction ratings : -0.208
```

```
# b
```

```
from scipy.stats import kurtosis
```

```
Kurtosis = kurtosis(ratings)
```

```
print(f"kurtosis of the satisfaction ratings : {Kurtosis:.3f}")
```

```
kurtosis of the satisfaction ratings : -0.768
```

```
# c
```

```
print(f"Skewness : {skewness:.3f}")
```

```
print(f"Kurtosis : {Kurtosis:.3f}")
```

```
if skewness>0:
```

```
    S_text = "Positively skewed"
```

```
elif skewness<0:
```

```
    S_text = "Negatively skewed"
```

```
else:
```

```
    S_text = "Symmetrical"
```

```
if Kurtosis>0:
```

```
    K_text = "Leptokurtic"
```

```
elif Kurtosis<0:
```

```
    K_text = "Platykurtic"
```

```
else:
```

```
    K_text = "Mesokurtic"
```

```
print("\n Interpretation: ")
```

```
print(f" The distribution is : {S_text}")
```

```
print(f" The distribution is : {K_text}")
```

```
Skewness : -0.208
```

```
Kurtosis : -0.768
```

```
Interpretation:
```

The distribution is : Negatively skewed
The distribution is : Platykurtic

Question 7

A research study wants to analyze the weight distribution of a sample of

individuals to assess their health and body composition.

Data: Let's consider the weights (in kilograms) of a sample of 100 individuals:

Weights:

55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 88, 90, 92, 95,
100, 105, 110, 115,

120, 125, 130, 135, 140, 145, 150, 155, 160, 165, 170, 175,
180, 185, 190, 195,

200, 205, 210, 215, 220, 225, 230, 235, 240, 245, 250, 255,
260, 265, 270, 275,

280, 285, 290, 295, 300, 305, 310, 315, 320, 325, 330, 335,
340, 345, 350, 355,

360, 365, 370, 375, 380, 385, 390, 395, 400, 405, 410, 415,
420, 425, 430, 435,

440, 445, 450, 455, 460, 465, 470, 475, 480, 485, 490,
495, 500, 505, 510, 515

Questions:

a. Quartiles: Calculate the first quartile (Q1), median (Q2),
and third quartile

(Q3) of the weight distribution.

b. Percentiles: Calculate the 15th percentile, 50th percentile, and 85th

percentile of the weight distribution.

c. Interpretation: Based on the quartiles and percentiles, what can be inferred about the weight distribution of the individuals?

```
Weights = np.array([55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85,
88, 90, 92, 95, 100, 105, 110, 115,
120, 125, 130, 135, 140, 145, 150, 155, 160, 165,
170, 175, 180, 185, 190, 195,
200, 205, 210, 215, 220, 225, 230, 235, 240, 245,
250, 255, 260, 265, 270, 275,
280, 285, 290, 295, 300, 305, 310, 315, 320, 325,
330, 335, 340, 345, 350, 355,
360, 365, 370, 375, 380, 385, 390, 395, 400, 405,
410, 415, 420, 425, 430, 435,
440, 445, 450, 455, 460, 465, 470, 475, 480, 485,
490, 495, 500, 505, 510, 515])
```

```
## a
```

```
Q1 = np.percentile(Weights,25)
Median_Q2 = np.median(Weights)
Q3 = np.percentile(Weights,75)
print(f" First Quartile : {Q1}")
print(f" Q2 : {Median_Q2}")
print(f" Third Quartile : {Q3}")
```

```
First Quartile : 143.75
Q2 : 267.5
Third Quartile : 391.25
```

```
## b
```

```
P15 = np.percentile(Weights,15)
P50 = np.percentile(Weights,50)
P85 = np.percentile(Weights,85)
print(f" 15th percentile : {P15}")
print(f" 50th percentile : {P50}")
print(f" 85th percentile : {P85:.2f}")
```

```
15th percentile : 94.55
50th percentile : 267.5
85th percentile : 440.75
```

```
## c
```

```
# Explanation
```

```
print("""
```

Explanation:

The median indicated is 267.5 kg. This suggests that half of individuals weigh more than 267.5 kg and half weigh less.

A high amount of variability is suggested in weights because of a wide interquartile range given in Q1 and Q3 ($391.25 - 143.75 = 247.5$ kg).

A small amount of individuals who weigh 15% of a population is indicated at 15th percentile of 94.55 kg.

The 85th percentile indicates that 15% of people are in the highest weight group. The 85th percentile is 440.75 kg which indicates that 85% of people are under 441 kg.

Given that the distance between the median and 85th percentile is large indicates that there are far more individuals with higher weight. This translates to a right skewed (or positively skewed) distribution.

```
""")
```

Explanation:

The median indicated is 267.5 kg. This suggests that half of individuals weigh more than 267.5 kg and half weigh less.

A high amount of variability is suggested in weights because of a wide interquartile range given in Q1 and Q3 ($391.25 - 143.75 = 247.5$ kg).

A small amount of individuals who weigh 15% of a population is indicated at 15th percentile of 94.55 kg.

The 85th percentile indicates that 15% of people are in the highest weight group. The 85th percentile is 440.75 kg which indicates that 85% of people are under 441 kg.

Given that the distance between the median and 85th percentile is large indicates that there are far more individuals with higher weight. This translates to a right skewed (or positively skewed) distribution.

Question 8

A researcher wants to examine the relationship between the hours spent studying

and the exam scores of a group of students.

Data: Let's consider the number of hours spent studying and the corresponding

exam scores for a sample of 30 students:

Hours Spent Studying: 10, 12, 15, 18, 20, 22, 25, 28, 30, 32, 35, 38, 40, 42, 45, 48,

50, 52, 55, 58, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82

Exam Scores: 60, 65, 70, 75,

80, 82, 85, 88, 90, 92, 93, 95, 96, 97, 98, 99, 100, 102, 105, 106, 107, 108, 110,

112, 114, 115, 116, 118, 120, 122

Question:

Calculate the correlation coefficient between the hours spent studying and the

exam scores. Interpret the value of the correlation coefficient and explain the

nature of the relationship between studying hours and exam scores

```
study_hrs = np.array([10, 12, 15, 18, 20, 22, 25, 28, 30, 32, 35, 38, 40, 42, 45, 48, 50, 52, 55, 58, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82])
```



```
Exam_scores = np.array([60, 65, 70, 75, 80, 82, 85, 88, 90, 92, 93, 95, 96, 97, 98, 99, 100, 102, 105, 106, 107, 108, 110, 112, 114, 115, 116, 118, 120, 122])
```

```
## Correlation Coefficient
```

```
Corr_Coef = np.corrcoef(study_hrs, Exam_scores)[0,1]
```

```
print(f"Correlation Coefficient : {Corr_Coef:.3f}")
```

```
## Interpretation
```

```
print("""
```

```
Interpretation:
```

```
The correlation coefficient between hours spent studying and exam scores is approximately 0.98.
```

```
This value is very close to +1, indicating a strong positive linear relationship.
```

```
This suggests that as the number of hours spent studying increases, exam scores also tend to increase.
```

```
Therefore, students who study more generally achieve higher scores in the exam.
```

```
""")
```

```
Correlation Coefficient : 0.977
```

```
Interpretation:
```

```
The correlation coefficient between hours spent studying and exam scores is approximately 0.98.
```

```
This value is very close to +1, indicating a strong positive linear relationship.
```

```
This suggests that as the number of hours spent studying increases, exam scores also tend to increase.
```

```
Therefore, students who study more generally achieve higher scores in the exam.
```

Question 9

A multiple-choice test consists of 10 questions, each with four possible answers. If a

student randomly guesses on each question, what is the probability of getting at least

8 questions correct? Data: Number of questions (n) = 10, Number of possible

answers per question (k) = 4

```
from scipy.stats import binom

## Total questions(n)
N = 10

## Since we have 4 options and out of that 1 option is the right answer then the probability of guessing that answer is 1/4
P = 1/4

## So here we have to find the probability of getting atleast 8 answers correctly(guessing) so we have to calculate for 8,9,10
## and here we use binomial distribution
prob = binom.pmf(8,N,P) + binom.pmf(9,N,P) + binom.pmf(10,N,P)

print(f"Probabilty of getting atleast 8 correct answers by guessing : {prob:.6f}")

Probabilty of getting atleast 8 correct answers by guessing : 0.000416
```

Question 10

The lifetimes of a certain brand of light bulbs are normally distributed with a mean

of 1000 hours and a standard deviation of 100 hours. What is the probability that a

randomly selected light bulb lasts between 900 and 1100 hours? Data: Mean lifetime

(μ) = 1000 hours, Standard deviation (σ) = 100 hours, Lifetime range (lower limit x1,

upper limit x2)

```
from scipy.stats import norm
## Data we have : Mean, stdev, range(lower and Upper)

mu = 1000
sigma = 100
low_limit = 900
up_limit = 1100

## probability of randomly selected light bulb that lasts between 900 and 1100
probability = norm.cdf(up_limit, mu, sigma) -
norm.cdf(low_limit, mu, sigma)

print(f"Probability that a light bulb lasts between 900 and 1100 hrs :
{probability:.3f}")
```

Probability that a light bulb lasts between 900 and 1100 hrs : 0.683

Question 11

A study is conducted to estimate the mean height of a population. A random sample

of 100 individuals is selected, and their heights are measured. Calculate a 95%

confidence interval for the population mean height, given that the sample mean

height is 170 cm and the sample standard deviation is 8 cm. Data: Sample size (n) =

100, Sample mean ($\bar{x}$) = 170 cm, Sample standard deviation (s) = 8 cm, Confidence

level = 95%

```
import math
from scipy import stats
n = 100 ## Sample Size
mean = 170
std_dev = 8 ## Standard Deviation
confidence = 0.95

SE = std_dev/math.sqrt(n) ## Standard Error

df = n-1 ## Degrees of freedom

t_critical = stats.t.ppf((1 + confidence) / 2, df) ## t-critical value
for 95% Confidence

ME = t_critical * SE ## Margin of error

## Confidence Interval

lower_bound = mean - ME
upper_bound = mean + ME

print(f"95% Confidence Interval: ({lower_bound:.2f},
{upper_bound:.2f}) cm")

95% Confidence Interval: (168.41, 171.59) cm
```

Question 12

A manufacturing company claims that the average weight of its product is 500

grams. To test this claim, a random sample of 25 products is selected, and their

weights are measured. The sample mean weight is found to be 510 grams with a

sample standard deviation of 20 grams. Perform a hypothesis test to determine if

there is evidence to support the company's claim. Data: Sample size (n) = 25,

Sample mean ($\bar{x}$) = 510 grams, Sample standard deviation (s) = 20 grams,

Population mean (μ) = 500 grams

```
import math
from scipy import stats

## Given Data

Sample_mean = 510
mu = 500 ## Population Mean
n = 25 ## Sample size
s = 20 ## Sample standard deviation
df = n-1 ## Degree of freedom
alpha = 0.05 ## Significance level

t_test = (Sample_mean - mu)/(s/math.sqrt(n))

t_critical = stats.t.ppf(1-alpha/2,df)

print(f"t_statistics : {t_test:.3f}")
print(f"critical t-value: +-{t_critical:.3f}")

## Decision
if t_test > t_critical:
```

```

    print("Reject the null hypothesis: The product weight is
significantly different from 500g.")
else:
    print("Fail to reject the null hypothesis: The company's claim is
supported.")

t_statistics : 2.500
critical t-value: +-2.064
Reject the null hypothesis: The product weight is significantly
different from 500g.

```

Question 13

A researcher wants to test whether a new teaching method improves student

performance. A random sample of 50 students is divided into two groups: one group

taught using the new method and the other using the traditional method. The average

test scores of the two groups are compared. State the null and alternative hypotheses

or this study. Data: Sample size (n) = 50, Test scores of the two groups

```

import math
from scipy.stats import ttest_ind

## Example data
import numpy as np
from scipy import stats

# Example data

new_method_scores = [78, 82, 85, 90, 88, 76, 84, 91, 79, 87,
                     83, 80, 89, 92, 77, 85, 88, 86, 84, 90,
                     81, 87, 88, 85, 89]

traditional_scores = [72, 75, 78, 74, 76, 70, 73, 77, 75, 74,
                     71, 76, 72, 73, 74, 75, 70, 72, 74, 73,
                     71, 75, 72, 74, 73]

```

```

# independent two-sample t-test (one-tailed)
t_stat, p_value_two_tailed = stats.ttest_ind(new_method_scores,
traditional_scores, equal_var=False)

# we want the one-tailed (new method > traditional), we divide p-
value by 2
p_value_one_tailed = p_value_two_tailed / 2

print(f"t-statistic: {t_stat:.3f}")
print(f"Two-tailed p-value: {p_value_two_tailed:.4f}")
print(f"One-tailed p-value (H1: new > traditional):
{p_value_one_tailed:.4f}")

alpha = 0.05
if p_value_one_tailed < alpha and t_stat > 0:
    print("Reject H0(Null Hypothesis)")
else:
    print("Fail to reject H0(Null Hypothesis) ")

t-statistic: 11.485
Two-tailed p-value: 0.0000
One-tailed p-value (H1: new > traditional): 0.0000
Reject H0(Null Hypothesis)

```

Question 14

A survey is conducted to estimate the proportion of people in a city who support a

particular policy. A random sample of 500 individuals is surveyed, and 320 of them

express support for the policy. Calculate a 90% confidence interval for the

population proportion, given the sample proportion. Data:
Sample size (n) = 500,

Number of successes (x) = 320, Confidence level = 90%

```

n = 500 ## Sample size
x = 320 ## Number of successes

```

```

## Sample proportion
Sp = (x/n)

## Z score for 90 % confidence level
z = 1.645

## Standard Error
se = math.sqrt(Sp*(1-Sp)/n)

## Confidence Interval
lower = Sp - z*se
Upper = Sp + z*se

print("Sample proportion :", Sp)
print("Standard error :", round(se,4))
print(f"90% confidence interval : {lower:.3f} , {Upper:.3f}")

Sample proportion : 0.64
Standard error : 0.0215
90% confidence interval : 0.605 , 0.675

```

Question 15

A company sells smartphones, and the number of defects per batch follows a

Poisson distribution with a mean of 2 defects. What is the probability of having

exactly 3 defects in a randomly selected batch? Data: Mean number of defects (λ) =

2, Number of defects (x) = 3

```

from scipy.stats import poisson

# Given data
lam = 2 # mean number of defects (Lambda)
x = 3 # number of defects

# Probability using Poisson distribution
prob = poisson.pmf(x, lam)

print(f"Probability of exactly {x} defects: {prob:.4f}")

Probability of exactly 3 defects: 0.1804

```